DTU

02451 Introduction to Machine Learning

Week 8: Artificial Neural Networks and Optimization

Bjørn Sand Jensen

24 March

DTU Compute, Technical University of Denmark

Today

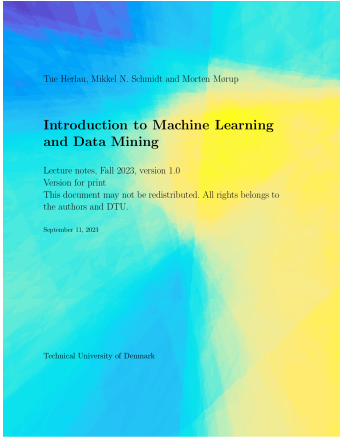
Feedback Groups of the day:

Elena Sofia Agostini, Markus Anker, Oliver Blumensaadt Bach, Sebastian Krener Balling, Mathias James Bibollet-Ruche, Jonas Idor Boklund, Eric Pierre Maurice Brevilliers, Alessandro Covi, Sebastian Philipp Ehmanns, Laetitia Finas, Matteo Franceschi, Asger Agesen Frimor, Frederik Nærvig Gerlach-Hansen, Arka Ghosh, Ly Nguyen Hansen, Jeppe Jensen Hedeby, Simone Frøslev Hoppe, Nora Johannessen, Hedi Khemakhem, Rasmus Kildegaard, Martin Knudsen, Jonas Jinlong Li, Mads-Frederik Frost Petersen, Elise Prahl-Larsen, Karl Martin Raid, Haoyang Sun, Togo Urayama, William Ziyu Ye, Qian Yu.

Reading/homework material:

Chapter 15

P15.1, P15.2, P15.3



Lecture Schedule

1. Introduction, data, and visualization
   3 Febuary: C1,C2, C7
2. Summary statistics, similarity, and nearest neighbors
   10 Febuary: C4, C12
3. Computational linear algebra and PCA
   17 Febuary: C3
4. Probability theory, Bayes and Naive Bayes
   24 Febuary: C5, C6, C13 **(Assignment 1-3 due on 26 February)**
5. Bayesian inference and linear models
   3 March: C5, C6, C8
6. Decision trees, overfitting and cross-validation
   10 March: C9, C10
7. Performance evaluation and AUC
   17 March: C11, C16 **(Assignment 4-6 due on 19 March)**
8. **Artificial Neural Networks and Optimization**
   24 March: C15
9. Bias/Variance and ensemble methods
   7 April: C14, 17
10. K-means and hierarchical clustering
    14 April: C18 **(Assignment 7-9 due on 16 April)**
11. Mixture models and density estimation
    21 April: C19, C20
12. Representation learning
    28 April: Material provided separately
13. Recap and discussion of the exam
    5 May: C1-C20 **(Assignment 10-12 due on 7 May)**

Online help: Piazza
Videos of lectures: `https://panopto.dtu.dk`
Streaming of lectures: Zoom (link on DTU Learn)

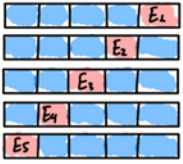
Practicalities and announcements

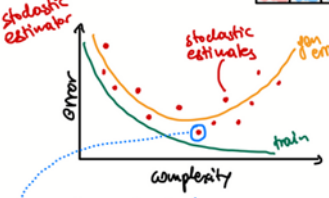
- Optional midterm quiz on DTU Learn - will not count towards your grade
- Midway survey - please contribute your input
- Week 8 Assignment (note more points than usual, but not greater weight in the final score).

Recap Lecture 7 (and 6)

Lecture 6

- Gen. error
$$E^{gen} = \mathbb{E}_{(x,y)}\left[L(t_{D}(x), y)\right]$$
- Estimate it with k-fold CV.
- $E^{gen} \approx \frac{1}{5}\sum_{k=1}^{5} E_k$





we risk to underestimate the error, i.e. be optimistic

IDEA: keep an extra test set to estimate the error of the selected model. Essentially 2 level CV.

Lecture 7

- Confidence interval: Instead of the estimation of the gen. error we the info you have to provide C.I as well.
- Assume that we use holdout CV

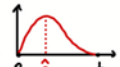
So we have for the test points

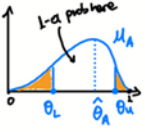
$$z_i = \begin{cases} 1, & \text{if } \hat{y}_i \neq y_i \\ 0, & \text{else} \end{cases}$$

$$p(z_1=1, z_2=0, \dots, z_{100}=1|\theta) = \overset{\text{iid assumption}}{\prod_{i=1}^{100} p(z_i|\theta)}$$

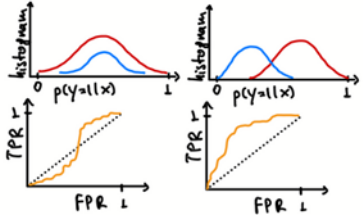
we model with Bernoulli

- if we consider a second model M_B





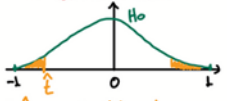
- ROC and AUC



- Hypothesis testing.

H_0: M_A and M_B same behavior

H_1: Different behavior



If $\hat{t}$ is unplausible under the H_0 then we reject it

Artificial Neural Networks

- for supervised machine learning

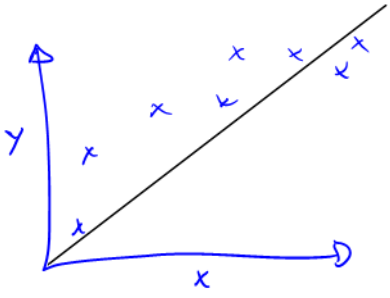
Recall: Linear models

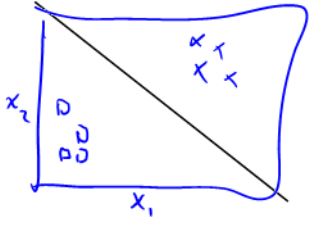
Remember the linear model?

- Regression
- Classification (logistic regression)

$$f(x; w) = w^T \tilde{x}$$

$$p(y = 1 | x, w) = \sigma(w^T \tilde{x})$$

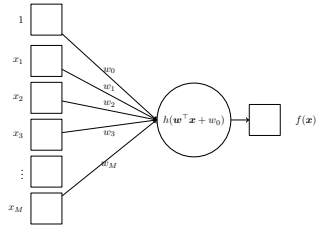


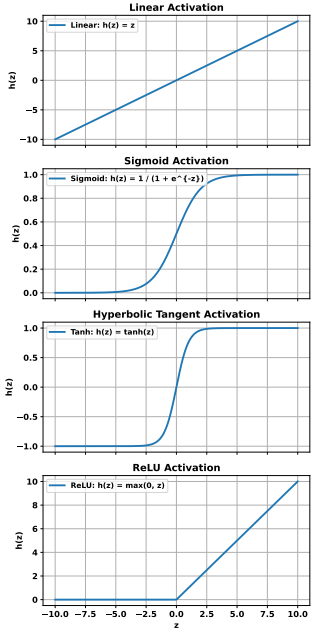


Generalized linear model

Remember the generalized linear model?

- Data $\{\boldsymbol{x}_i, y_i\}_{i=1}^N$
- Model $f(\boldsymbol{x}) = h(\tilde{\boldsymbol{x}}^\top \boldsymbol{w})$
- Loss function $d(y, f(\boldsymbol{x}))$
- Parameters $\boldsymbol{w}^* = \underset{\boldsymbol{w}}{\operatorname{argmin}} \sum_{n=1}^N d(y_i, f(\boldsymbol{x}_i))$





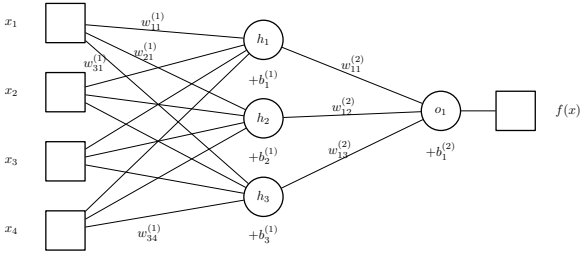
Generalized linear models

- **Pros:**
  - Interpretable and transparent
  - Efficient on small datasets
  - Easy to implement and validate
  - Strong baseline performance
- **Cons**
  - Limited ability to model complex patterns
  - Depend heavily on feature engineering to select the basic expansions/transformations
  - Less effective on images, text, and speech
  - Hard to scale to richer representations

Idea: Use the generalised linear model as a building block to create composite functions, resulting in a more flexible function class.

Artificial neural networks

- Data $\{\boldsymbol{x}_i, y_i\}_{i=1}^N$
- Model
  $f(\boldsymbol{x}) = h^{(2)}\left(b_1^{(2)} + \sum_{j=1}^H w_{1j}^{(2)} h^{(1)}\left(\boldsymbol{x}^\top \boldsymbol{w}_j^{(1)} + b_j^{(1)}\right)\right)$
- Loss function $d(y, f(\boldsymbol{x}))$
- Parameters $\boldsymbol{w}^* = \underset{\boldsymbol{w}}{\operatorname{argmin}} \sum_{n=1}^N d(y_i, f(\boldsymbol{x}_i))$



Example:

$$h^{(1)}(x) = \tanh(x)$$

$$h^{(2)}(x) = x$$

$$d(y, f(\boldsymbol{x})) = (y - f(\boldsymbol{x}))^2$$

Artificial neural networks

Feed forward neural network

- Each “neuron”
  - Computes a nonlinear function of the sum of its inputs
  - Is just like a generalized linear model
  - Has its own set of parameters
- Modeling choices
  - Cost function
  - Non-linearities
  - Number of neurons and hidden layers
  - Selection of inputs
- Parameter estimation using numerical optimization methods
- Very flexible model: can easily overfit

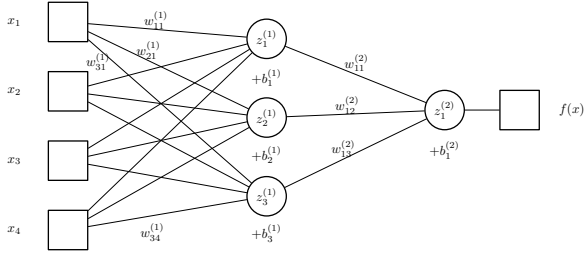
Neurons and layers

Recall:

$$f(\boldsymbol{x}) = h^{(2)}\left(b_1^{(2)} + \sum_{j=1}^{H} w_{1j}^{(2)} h^{(1)}\left(\boldsymbol{x}^\top \boldsymbol{w}_j^{(1)} + b_j^{(1)}\right)\right)$$

- Let $z_j^{(1)}$ be output of j'th hidden unit $z_j^{(1)} = h^{(1)}\left({\boldsymbol{w}_j^{(1)}}^\top \boldsymbol{x} + b_j^{(1)}\right)$
- For all the hidden neuros $\boldsymbol{z}^{(1)} = h^{(1)}\left(\boldsymbol{W}^{(1)}\boldsymbol{x} + b_j^{(1)}\right)$
- Output: $f(\boldsymbol{x}) = h^{(2)}\left(b_1^{(2)} + \sum_{j=1}^{H} w_{1j}^{(2)} z_j^{(1)}\right) = h^{(2)}\left(\boldsymbol{W}^{(2)}\boldsymbol{z}^{(1)} + b_j^{(1)}\right)$

$$= h^{(2)}\left(\boldsymbol{W}^{(2)} h^{(1)}\left(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\right) + \boldsymbol{b}^{(2)}\right)$$

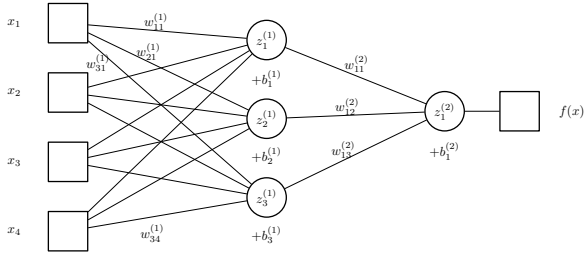


Neurons and layers

$$f(\boldsymbol{x}) = h^{(2)}\left(\boldsymbol{W}^{(2)} h^{(1)}\left(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\right) + \boldsymbol{b}^{(2)}\right)$$

$$\boldsymbol{W}^{(1)} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} & w_{14}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} & w_{24}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} & w_{33}^{(1)} & w_{34}^{(1)} \end{bmatrix}, \qquad \boldsymbol{b}^{(1)} = \begin{bmatrix} b_1^{(1)}, b_2^{(1)}, b_3^{(1)} \end{bmatrix}^\top$$

$$\boldsymbol{W}^{(2)} = \begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} \end{bmatrix}, \qquad \boldsymbol{b}^{(2)} = \begin{bmatrix} b_1^{(2)} \end{bmatrix}^\top$$

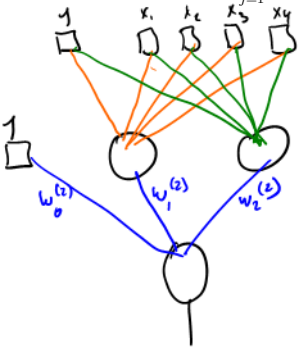


Quiz 1, Artificial Neural Network (Fall 2017)

13:52

We will consider an artificial neural network (ANN) trained to predict the average score of a player (i.e., y). The ANN is based on the model:

$$f(\boldsymbol{x}, \boldsymbol{w}) = w_0^{(2)} + \sum_{j=1}^{2} w_j^{(2)} h^{(1)}([1\ \boldsymbol{x}]\boldsymbol{w}_j^{(1)}).$$



where $h^{(1)}(x) = \max(x, 0)$ is the rectified linear function used as activation function in the hidden layer (i.e., positive values are returned and negative values are set to zero). We will consider an ANN with two hidden units in the hidden layer defined by:

$$\boldsymbol{w}_1^{(1)} = \begin{bmatrix} 21.78 \\ -1.65 \\ 0 \\ -13.26 \\ -8.46 \end{bmatrix}, \boldsymbol{w}_2^{(1)} = \begin{bmatrix} -9.60 \\ -0.44 \\ 0.01 \\ 14.54 \\ 9.50 \end{bmatrix},$$

and $w_0^{(2)} = 2.84$, $w_1^{(2)} = 3.25$, and $w_2^{(2)} = 3.46$. What is the predicted average score of a basketball player with observation vector $\boldsymbol{x}^* = [6.8\ 225\ 0.44\ 0.68]$?

A. 1.00

B. 3.74

C. 8.21

D. 11.54

E. Don't know.

The output is given by:

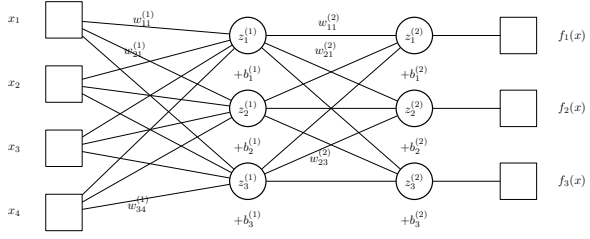
$$
\begin{aligned}
f(\boldsymbol{x}, \boldsymbol{w}) &= 2.84 \\
&\quad + 3.25 \cdot \max([1\ 6.8\ 225\ 0.44\ 0.68] \cdot \begin{bmatrix} 21.78 \\ -1.65 \\ 0 \\ -13.26 \\ -8.46 \end{bmatrix}, 0) \\
&\quad + 3.46 \max([1\ 6.8\ 225\ 0.44\ 0.68] \cdot \begin{bmatrix} -9.60 \\ -0.44 \\ 0.01 \\ 14.54 \\ 9.50 \end{bmatrix}, 0) \\
&= 2.84 + 3.25 \cdot \max(-1.027, 0) + 3.46 \max(2.516, 0) \\
&= 11.54
\end{aligned}
$$

Generalization 1: Multiple outputs

- As before define: $\boldsymbol{z}^{(1)} = h^{(1)}\left(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\right)$
- Now let $\boldsymbol{W}^{(2)}$ be a $C \times H$ matrix then:

$$\boldsymbol{y} = \boldsymbol{f}(\boldsymbol{x}) = h^{(2)}\left(\boldsymbol{W}^{(2)}\boldsymbol{z}^{(1)} + \boldsymbol{b}^{(2)}\right)$$

  will be C-dimensional
- Re-define error function $E(\boldsymbol{w}) = \sum_{i=1}^{N} \|\boldsymbol{y}_i - \boldsymbol{f}(\boldsymbol{x}_i)\|_2^2$

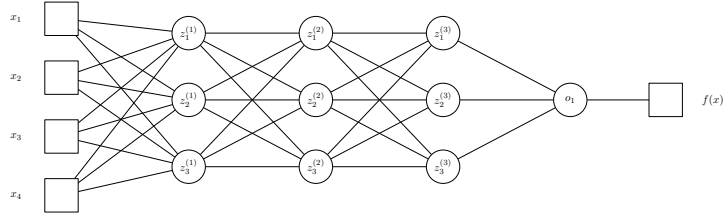


Generalization 2: Multiple layers (aka Deep Learning)

- Define $\boldsymbol{z}^{(0)} = \boldsymbol{x}$
- For each layer $l = 1, \ldots, L$ compute

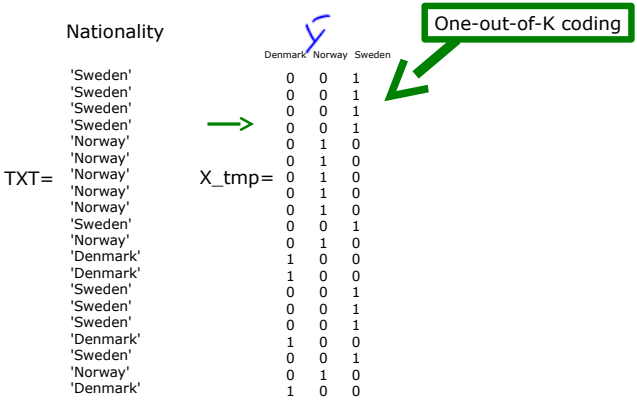
$$\boldsymbol{z}^{(l)} = h^{(l)}\left(\boldsymbol{W}^{(l)}\boldsymbol{z}^{(l-1)} + \boldsymbol{b}^{(l)}\right)$$

- Output is simply $\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{z}^{(L)}$



(no weigths labels and biases shown)

Single and multi-class: One out of K coding



Multi-class classification

- Logistic regression, $y = 0, 1$:

$$p(y|\theta) = \theta^y (1-\theta)^{1-y}$$

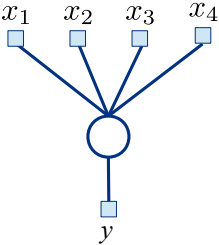
$$\theta = \sigma(\boldsymbol{x}^\top \boldsymbol{w})$$

$y=1 \Rightarrow \theta$

$y=0 \Rightarrow 1-\theta$

BCE

$$E(w) = \sum_i^N -y_i \log \theta_i - (1-y_i)\log(1-\theta_i)$$



- Multinomial regression, $y = 1, 2, \ldots, K$

$\bar{\boldsymbol{y}}$: one-of-K encoding of y, [0 0 1]

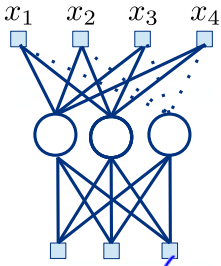
$$p(y|\boldsymbol{\theta}) = \prod_{k=1}^{K} \theta_k^{\bar{y}_k}$$

$\theta_1^0 \theta_2^0 \theta_3^1 = \theta_3^1$

$$\boldsymbol{\theta} = \text{softmax}\left(\begin{bmatrix}\boldsymbol{x}^\top \boldsymbol{w}_1 & \cdots & \boldsymbol{x}^\top \boldsymbol{w}_K\end{bmatrix}\right)$$

$$= \begin{bmatrix} \frac{e^{\boldsymbol{x}^\top \boldsymbol{w}_1}}{\sum_{c=1}^K e^{\boldsymbol{x}^\top \boldsymbol{w}_c}} & \cdots & \frac{e^{\boldsymbol{x}^\top \boldsymbol{w}_{K-1}}}{\sum_{c=1}^K e^{\boldsymbol{x}^\top \boldsymbol{w}_c}} & \frac{e^{\boldsymbol{x}^\top \boldsymbol{w}_K}}{\sum_{c=1}^K e^{\boldsymbol{x}^\top \boldsymbol{w}_c}} \end{bmatrix}$$

or: $$\boldsymbol{\theta} = \begin{bmatrix} \frac{e^{\boldsymbol{x}^\top \boldsymbol{w}_1}}{1+\sum_{c=1}^{K-1} e^{\boldsymbol{x}^\top \boldsymbol{w}_c}} & \cdots & \frac{e^{\boldsymbol{x}^\top \boldsymbol{w}_{K-1}}}{1+\sum_{c=1}^{K-1} e^{\boldsymbol{x}^\top \boldsymbol{w}_c}} & \frac{1}{1+\sum_{c=1}^{K-1} e^{\boldsymbol{x}^\top \boldsymbol{w}_c}} \end{bmatrix}$$



Connection to neural networks

Multinomial regression:

- Define:

$$\boldsymbol{\theta} = \begin{bmatrix} \frac{e^{\boldsymbol{x}^\top \boldsymbol{w}_1}}{\sum_{c=1}^{K} e^{\boldsymbol{x}^\top \boldsymbol{w}_c}} & \cdots & \frac{e^{\boldsymbol{x}^\top \boldsymbol{w}_K}}{\sum_{c=1}^{K} e^{\boldsymbol{x}^\top \boldsymbol{w}_c}} \end{bmatrix}$$

- Cost function is ($z_{i\cdot}$ is one-of-K encoding of y_i)

$$E(\boldsymbol{w}) = -\sum_{i=1}^{N} \log p(y_i|\boldsymbol{x}_i) = -\sum_{i=1}^{N}\sum_{k=1}^{K} \bar{y}_{ik} \log \theta_{ik}$$

Multi-class neural network:

- Suppose $z_1, \ldots, z_H$ are outputs of a neural network
- Define

$$\boldsymbol{\theta} = \begin{bmatrix} \frac{e^{\boldsymbol{z}^\top \boldsymbol{w}_1}}{\sum_{c=1}^{K} e^{\boldsymbol{z}^\top \boldsymbol{w}_c}} & \cdots & \frac{e^{\boldsymbol{z}^\top \boldsymbol{w}_K}}{\sum_{c=1}^{K} e^{\boldsymbol{z}^\top \boldsymbol{w}_c}} \end{bmatrix}$$

- Cost function is:

$$E(\boldsymbol{w}) = -\sum_{i=1}^{N} \log p(y_i|\boldsymbol{z}_i) = -\sum_{i=1}^{N}\sum_{k=1}^{K} \bar{y}_{ik} \log \theta_{ik}$$

14:30

Quiz 2, Multinomial Regression (Spring 2016)

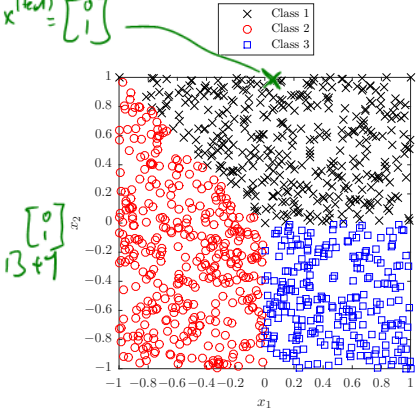


Figure 1: Observations labelled with the most probable class

Consider a multinomial regression classifier for a three-class problem where for each point $\boldsymbol{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}^\top$ we compute the class-probability using the softmax function

$$P(\hat{y} = k) = \frac{e^{\boldsymbol{w}_k^\top \boldsymbol{x}}}{e^{\boldsymbol{w}_1^\top \boldsymbol{x}} + e^{\boldsymbol{w}_2^\top \boldsymbol{x}} + e^{\boldsymbol{w}_3^\top \boldsymbol{x}}}.$$

A dataset of $N = 1000$ points where each point is labeled according to the maximum class-probability is shown in Figure 1. Which setting of the weights was used?

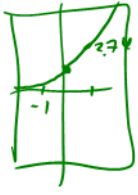
A. $w_1 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$, $w_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $w_3 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

B. $w_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$, $w_2 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$, $w_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

C. $w_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $w_2 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$, $w_3 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

D. $w_1 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$, $w_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $w_3 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$

E. Don't know.

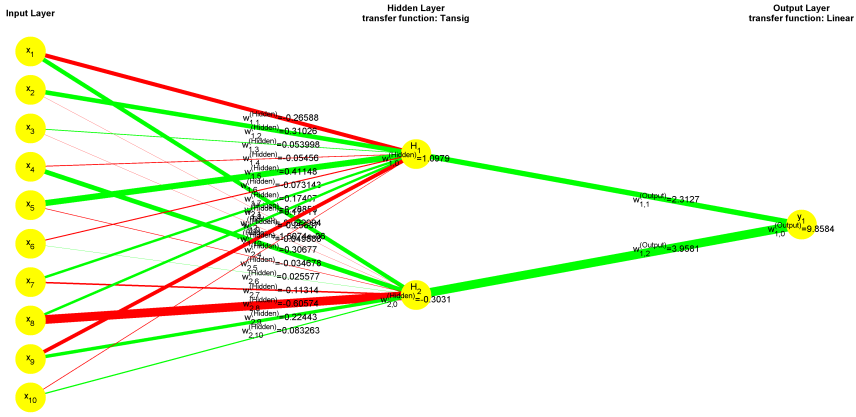


Consider for instance the point $\boldsymbol{x}$ where $x_1 = 0$ and $x_2 = 1$. Then, letting $y_k = \boldsymbol{w}_k^T \boldsymbol{x}$, we obtain:

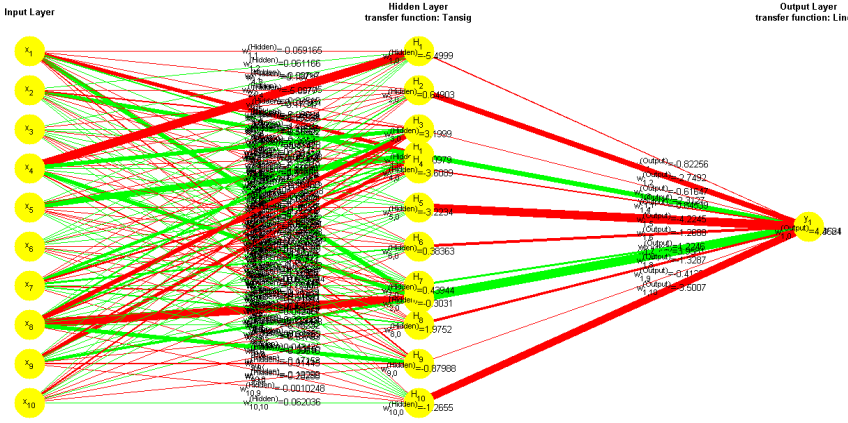
$$
\begin{aligned}
A &: \begin{bmatrix} y_1 & y_2 & y_3 \end{bmatrix} = \begin{bmatrix} -1 & 1 & -1 \end{bmatrix} \\
B &: \begin{bmatrix} y_1 & y_2 & y_3 \end{bmatrix} = \begin{bmatrix} -1 & -1 & 1 \end{bmatrix} \\
C &: \begin{bmatrix} y_1 & y_2 & y_3 \end{bmatrix} = \begin{bmatrix} 1 & -1 & -1 \end{bmatrix} \\
D &: \begin{bmatrix} y_1 & y_2 & y_3 \end{bmatrix} = \begin{bmatrix} -1 & 1 & 1 \end{bmatrix}
\end{aligned}
$$

Next, since the multinomial regression function preserves order we need only consider the maximal value. Accordingly the point $\boldsymbol{x}$ is only classified to the correct class 1 for option C.

Interpreting neural networks can be difficult



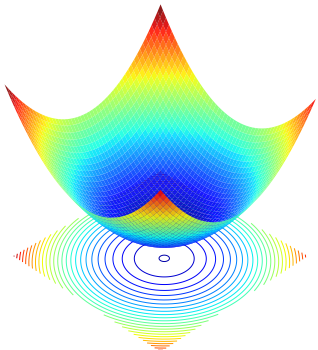
Interpreting neural networks can be difficult



Optimization

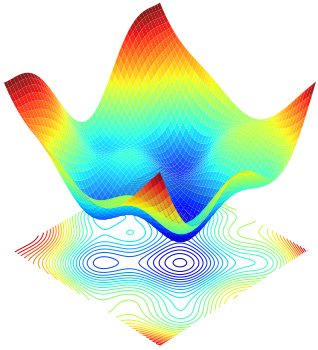
Recall: linear models

- Data: $\{\boldsymbol{x}_i, y_i\}_{i=1}^N$ where $\boldsymbol{x}_i \in \mathbb{R}^D$ and $y_i \in \mathbb{R}$.
- Model: $f(\boldsymbol{x}; \boldsymbol{w}) = \boldsymbol{w}^\top \phi(\boldsymbol{x})$ for $\phi : \mathbb{R}^D \to \mathbb{R}^H$ and $\boldsymbol{w} \in \mathbb{R}^H$.
- Loss function: $E(\boldsymbol{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - f(\boldsymbol{x}_i; \boldsymbol{w}))^2$
- Critical points: $\nabla_{\boldsymbol{w}} E(\boldsymbol{w}) = 0 \quad \Rightarrow \quad \boldsymbol{w}^* = (\boldsymbol{\Phi}^\top \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^\top \boldsymbol{y}$
- What about logistic regression?



Back to ANNs

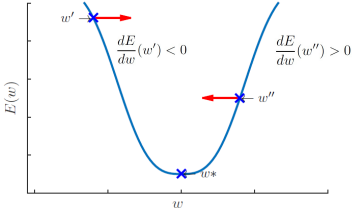
- Data: $\{\boldsymbol{x}_i, y_i\}_{i=1}^N$ where $\boldsymbol{x}_i \in \mathbb{R}^D$ and $y_i \in \mathbb{R}$.
- Model: $f(\boldsymbol{x}; \boldsymbol{w}) = h^{(2)}(\boldsymbol{W}^{(2)}\ h^{(1)}(\boldsymbol{W}^{(1)}\ \boldsymbol{x} + \boldsymbol{b}^{(1)}) + \boldsymbol{b}^{(2)})$ for $\boldsymbol{w} = \{\boldsymbol{W}^{(1)}, \boldsymbol{b}^{(1)}, \boldsymbol{W}^{(2)}, \boldsymbol{b}^{(2)}\}$ and activations $h^{(1)}, h^{(2)}$.
- Loss function: $E(\boldsymbol{w}) = \frac{1}{N}\sum_{i=1}^N (y_i - f(\boldsymbol{x}_i; \boldsymbol{w}))^2$
- Critical points: $\nabla_{\boldsymbol{w}} E(\boldsymbol{w}) = 0 \quad \Rightarrow$ **Not so easy!**
- **Issues:** Non-convexity / multiple-local minma

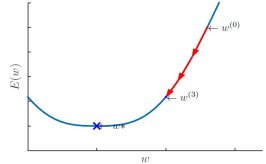


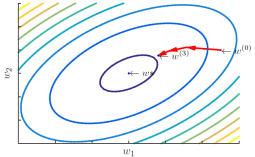
Basic numerical optimizer: Gradient Descent

– Start from an initial guess $\boldsymbol{w}^{(0)}$ for the optimal $\boldsymbol{w}^*$
– At step t of the algorithm, modify $\boldsymbol{w}^{(t)}$ to produce a better guess $\boldsymbol{w}^{(t+1)}$

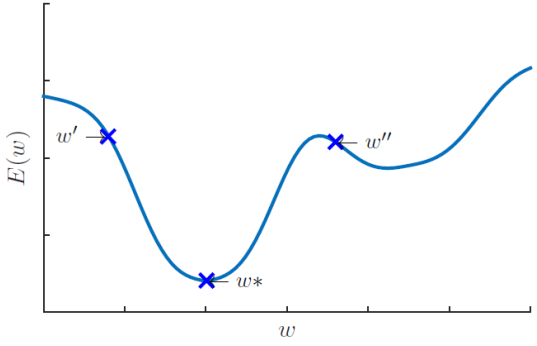
$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \varepsilon \frac{dE}{d\boldsymbol{w}}(\boldsymbol{w}^{(t)})$$







Contrary to least-squares linear regression and logistic regression, ANNs have issues of local minima



Some (of the many) variants

- Gradient Descent with Momentum
  - Use previous gradient to smooth and accelerate the trajectory.
  - Can help to escape shallow local minima, but better techniques exist.

$$\begin{aligned} \boldsymbol{z}^{(t+1)} &= \beta \boldsymbol{z}^{(t)} + \nabla_{\boldsymbol{w}} E(\boldsymbol{w}^{(t)}) \\ \boldsymbol{w}^{(t+1)} &= \boldsymbol{w}^{(t)} - \varepsilon \boldsymbol{z}^{(t+1)} \end{aligned} \quad \Leftrightarrow \quad \boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \varepsilon \left[\sum_{j=0}^{t} \beta^j \nabla_{\boldsymbol{w}} E(\boldsymbol{w}^{(t-j)}) \right]$$

- Stochastic Gradient Descent
  - Create a *batch* $B^{(t)}$ of size M by sub-sampling the full dataset.
  - Compute the gradient over the batch and update the parameters:

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \varepsilon \frac{1}{M} \sum_{i \in B^{(t)}} \nabla_{\boldsymbol{w}} d(y_i, f(\boldsymbol{x}_i; \boldsymbol{w}^{(t)}))$$

- Adaptive techniques: Adam, Adagrad, RMSprop, etc.
- Second order optimizers: Newton's method, Natural gradient, L-BFGS, etc.
- And many more...

The gradients for an ANN (and back-propagation)

- **Manually** computing the gradient (using rules of differentiation, in particular the chain rule)
  - **Issues**: Need gradients derived (pen & paper) and implemented (correctly) for every configuration of your network (of course, some reusable elements), which limits flexibility.
- **Automatically** using automatic differentiation techniques on a computational graph
  - PyTorch, Tensorflow, JAX, etc.

In this course: PyTorch

Summary

- Neural networks
- Multioutput models (linear and neural networks)
- Optimization

Resources

https://www.youtube.com Exellent video resource explaining the concepts behind neural networks

(https://www.youtube.com/watch?v=aircAruvnKk&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi)

http://playground.tensorflow.org Sleek interactive neural network example where you can examine the effect of different number of hidden neurons, activation functions, and many other things on training

(http://playground.tensorflow.org/)

https://www.tensorflow.org A popular and well-documented deep learning framework. While well documented, notice it requires some python knowledge. Used in industry and production settings.

(https://www.tensorflow.org/)

https://pytorch.org Most popular and well-documented deep learning framework. In some ways slightly simpler framework for deep learning. Dominant in research and for prototyping.

(https://pytorch.org/)